This will be regading DSA learning and gaining mastery over Arrays and Strings and HashMaps 🔥 DAY 1
🟦 Arrays Concept: HashMap Basics + Two Sum Pattern

Problems:

Two Sum

Contains Duplicate

🟥 Strings Concept: Frequency Map

Problems:

Valid Anagram

First Unique Character in a String

🏆 Classical Problem:

Aggressive Cows (Binary Search on Answer)

First lets start of with Arrays : Teach me this : 🟦 Arrays Concept: HashMap Basics + Two Sum Pattern
Then python implementation for these :Two Sum, Contains Duplicate

Love the energy Gagan 🔥
This is the correct way to build DSA mastery — concept first, then patterns, then problems.

Today we build a **foundational weapon**:

🟦 Arrays Concept: HashMap Basics + Two Sum Pattern

This pattern alone appears in:

- Trading coding rounds
- FAANG interviews
- Online assessments
- Competitive programming

🔵 1. What is a HashMap? (Core Idea)

In Python → **Dictionary (`dict`)**

It stores:

```
key → value
```

Why is it powerful?

Because lookup time is:

```
O(1) average time
```

That means:

- Check if element exists → O(1)
- Insert element → O(1)
- Update frequency → O(1)

This is why it beats brute force solutions.

🔥 2. The Two Sum Pattern (VERY IMPORTANT)

❌ Brute Force Idea

For every element:

- Check all other elements
- Time complexity = $O(n^2)$

Bad for large inputs.

✅ Optimized Thinking (HashMap Pattern)

Instead of asking:

> "Does this number have a pair?"

We ask:

> "Have I already seen the number that I need?"

💡 Core Logic

If:

```
a + b = target
```

Then:

So for each number:

1. Compute required = target - current_number
2. Check if required exists in HashMap
3. If yes → DONE
4. If not → store current number in HashMap

🧠 Pattern Template (Memorize This)

```
Initialize empty hashmap

For each element:
    Find required value
    If required in hashmap:
        return answer
    Else:
        store current element in hashmap
```

This is called:

🚀 Complement Search Pattern

Used in:

- Two Sum
- Subarray Sum Equals K
- Pair with given difference
- Many sliding window problems

🟦 Problem 1: Two Sum

Problem

Given array `nums` and integer `target`, return indices of two numbers that add up to target.

✅ Python Implementation

Python

Run

```python
def twoSum(nums, target):
    hashmap = {}  # value -> index

    for i in range(len(nums)):
        required = target - nums[i]

        if required in hashmap:
            return [hashmap[required], i]

        hashmap[nums[i]] = i

    return []
```

🔍 Example Walkthrough

```
nums = [2,7,11,15]
target = 9
```

Step 1:
i=0 → value=2
required = 7
7 not in hashmap
store {2:0}

Step 2:
i=1 → value=7
required = 2
2 exists → return [0,1]

Boom 🔥

Time Complexity: O(n)
Space Complexity: O(n)

🟦 Problem 2: Contains Duplicate

Problem

Return True if any value appears at least twice.

🧠 Thinking

If we see a number again → duplicate found.

So:

- Maintain HashSet (or dict)
- If number already exists → return True

✅ Python Implementation

Method 1: Using Set (Cleanest)

Python Run

```python
def containsDuplicate(nums):
    seen = set()

    for num in nums:
        if num in seen:
            return True
        seen.add(num)

    return False
```

Method 2: Using HashMap

Python Run

```python
def containsDuplicate(nums):
    hashmap = {}

    for num in nums:
        if num in hashmap:
            return True
        hashmap[num] = 1

    return False
```

🔍 Example

```
nums = [1,2,3,1]
```

1 → store
2 → store

3 → store
1 → already exists → True

🔥 Important Interview Insight

Why does this work fast?

Because Python dictionary uses:

```
Hashing
```

Which converts keys into an index internally.

That's why lookup is O(1).

🧠 Gagan's Mastery Rule for Today

To truly master this:

1. Code without looking
2. Dry run on paper
3. Solve variations:
   - Two Sum II (sorted array)
   - Two Sum returning boolean
   - Return number of pairs

🚀 Next Step (After You Practice)

We will move to:

🟥 Strings Concept: Frequency Map
→ Valid Anagram
→ First Unique Character

Then
🏆 Aggressive Cows (Binary Search on Answer — next level thinking)

Now tell me:

Do you want:

- 5 practice variations on Two Sum pattern?
- Or move to Strings Frequency Map?

Let's build this like a quant 🔥

Provide me an indetail walkthrough taking a better example and showing the complete dry run of the TWO SUM problem along with the code given

Perfect Gagan 🔥
Today we go **deep** — not surface level.

We'll do:

1. ✅ Clear problem understanding
2. ✅ Better example (with negative numbers + non-adjacent solution)
3. ✅ Full dry run (iteration by iteration table)
4. ✅ Visual intuition
5. ✅ Code explanation line-by-line
6. ✅ Why this works (core insight)

🟦 TWO SUM — Deep Dive

🧩 Problem

Given:

```
nums = [4, -2, 1, 7, 5, 3]
target = 6
```

Find indices of two numbers such that:

```
nums[i] + nums[j] = 6
```

🔍 Step 0: Brute Force Thinking (Wrong Direction)

You might think:

Check all pairs:

```
4 + -2
4 + 1
```

```
4 + 7
...
```

That's $O(n^2)$.

We want O(n).

🔥 Core Insight

Instead of asking:

> "Which pair gives 6?"

We ask:

> "For this number, what do I need to reach 6?"

That value is:

```
required = target - current_number
```

🧠 Algorithm Strategy

- Create empty hashmap
- Traverse array once
- For each element:
  - Calculate required
  - Check if required already exists
  - If yes → return answer
  - If no → store current number

🟦 The Code (Same As Before)

Python Run

```python
def twoSum(nums, target):
    hashmap = {}  # value -> index

    for i in range(len(nums)):
        required = target - nums[i]

        if required in hashmap:
            return [hashmap[required], i]
```

```
        hashmap[nums[i]] = i

    return []
```

🔥 FULL DRY RUN (VERY IMPORTANT)

Input

```
nums = [4, -2, 1, 7, 5, 3]
target = 6
```

We maintain:

```
hashmap = {}
```

🔁 Iteration 1

i = 0

nums[i] = 4

```
required = 6 - 4 = 2
```

Check:

Is `2` in hashmap?

❌ No

So store:

```
hashmap = {4: 0}
```

🔁 Iteration 2

i = 1

nums[i] = -2

```
required = 6 - (-2)
required = 8
```

Check:

Is `8` in hashmap?

❌ No

Store:

```
hashmap = {
    4: 0,
    -2: 1
}
```

🔁 Iteration 3

i = 2

nums[i] = 1

```
required = 6 - 1 = 5
```

Check:

Is `5` in hashmap?

❌ No

Store:

```
hashmap = {
    4: 0,
    -2: 1,
    1: 2
}
```

🔁 Iteration 4

i = 3

nums[i] = 7

```
required = 6 - 7 = -1
```

Check:

Is `-1` in hashmap?

❌ No

Store:

```
hashmap = {
    4: 0,
    -2: 1,
    1: 2,
    7: 3
}
```

🔁 Iteration 5

i = 4

nums[i] = 5

```
required = 6 - 5 = 1
```

Check:

Is `1` in hashmap?

✅ YES

Hashmap says:

```
1 → index 2
```

So we return:

```
[2, 4]
```

Meaning:

```
nums[2] + nums[4]
1 + 5 = 6
```

🎯 DONE.

🔥 Important Observations

Notice:

We never checked:

- 5 + 4
- 5 + -2
- 5 + 7

Why?

Because we already stored previous elements.

We are always asking:

> "Did I already see what I need?"

🧠 Why We Check Before Storing

Imagine this case:

```
nums = [3,3]
target = 6
```

Iteration 1:
store {3:0}

Iteration 2:
required = 3
3 exists → return [0,1]

If we stored first and checked after, logic would break in some variations.

Always:

```
Check first
Then store
```

📊 Visual Table Summary

i	nums[i]	required	hashmap before	Action
0	4	2	{}	store 4
1	-2	8	{4:0}	store -2

i	nums[i]	required	hashmap before	Action
2	1	5	{4,-2}	store 1
3	7	-1	{4,-2,1}	store 7
4	5	1	{4,-2,1,7}	FOUND

🟦 Line-by-Line Code Understanding

Python Run

```python
hashmap = {}
```

Stores:

```
number → index
```

Python Run

```python
for i in range(len(nums)):
```

Traverse once → O(n)

Python Run

```python
required = target - nums[i]
```

Compute complement

Python Run

```python
if required in hashmap:
```

O(1) lookup

Python Run

```
return [hashmap[required], i]
```

Return indices

Python Run

```
hashmap[nums[i]] = i
```

Store current element for future use

🚀 Complexity

Time: O(n)
Space: O(n)

This is optimal.

🧠 Interview-Level Insight

This pattern is actually:

> Searching previous prefix efficiently

It's a **stream processing mindset**.

Very powerful in:

- Subarray sum
- Prefix sums
- Frequency counting
- Sliding window

🏆 Gagan Mastery Exercise (Do This Now)

Without looking:

1. Dry run:

```
nums = [8, 2, 6, 3, 9]
target = 11
```

2. Modify code to return True/False instead of indices.
3. Try case:

```
nums = [1,5,1,5]
target = 10
```

Now tell me:

Do you want next:

🟥 Frequency Map (Strings) deep dive
OR
Harder Two Sum variations (sorted array + duplicates + multiple pairs)

We build this like a quant brain 🔥

Modify code to return True/False instead of indices.Will this be correct ??

```
def twoSum(nums, target):
    hashmap = {}  # value -> index

    for i in range(len(nums)):
        required = target - nums[i]

        if required in hashmap:
            return True

        hashmap[nums[i]] = i

    return False
```

Yes Gagan 🔥
Your logic is **100% correct** for the Boolean version.

Let's analyze it properly like an interviewer would.

✅ Your Code

```
def twoSum(nums, target):
    hashmap = {}  # value -> index
```

```
    for i in range(len(nums)):
        required = target - nums[i]

        if required in hashmap:
            return True

        hashmap[nums[i]] = i

    return False
```

🧠 Is It Correct?

✔ YES — for the question:

> "Return True if any two numbers sum to target."

It works perfectly.

🔍 Let's Dry Run Quickly

Example 1

```
nums = [8, 2, 6, 3, 9]
```

Python Run

Step-by-step:

i	nums[i]	required	hashmap	Result
0	8	3	{}	store 8
1	2	9	{8}	store 2
2	6	5	{8,2}	store 6
3	3	8	{8,2,6}	FOUND → True

Correct.

Example 2 (Duplicates)

```
nums = [3,3]
```

Python Run

i	nums[i]	required	hashmap	Result
0	3	3	{}	store 3
1	3	3	{3}	FOUND

Works perfectly.

🔥 Important Question

Do we even need index?

No.

Since we are not returning indices, we can simplify further.

🚀 Cleaner Version (Professional Style)

Since index is unused, we should use a **set instead of hashmap**.

Because we only care about existence.

```
def twoSum(nums, target):
    seen = set()

    for num in nums:
        required = target - num

        if required in seen:
            return True

        seen.add(num)

    return False
```

🧠 Why This Is Better

- Cleaner

- More readable
- Intent is clearer
- Slightly more memory efficient

In interviews, this is preferred.

🔥 Complexity

Time: O(n)
Space: O(n)

Optimal.

⚠️ Important Edge Case Understanding

Consider:

```
nums = [5]
```

Python Run

Loop runs once:

- required = 5
- not in seen
- add 5
- loop ends
- return False

Correct.

🧠 Interviewer Might Ask You

“What if array is sorted?”

Then better approach is:

👉 Two pointers
👉 O(1) space

But this solution works for any order.

🎯 Final Verdict

Your logic:

✅ Correct
✅ Efficient
✅ Interview ready

Now I'll challenge you slightly deeper 🔥

What if I ask:

> Return number of distinct pairs that sum to target?

Would your current logic work?

Or would we need modification?

Think carefully before answering.

To return the number of distinct pairs that sum to target ... current logic meaning the hashmap logic will still work for it .... i think beacuse while learning of performing dry run we stopped after getting the perfect two sum pair ... but the loop hasnt finished completion .... so it will still run over other iterations also and since we are returning as a list we can use another list named as result and can append the result into the result list :
so after:
if required in hashmaps:
result.append([hashmaps[required],i])

and finally return the results : return result
?? will this work ... can this be a good answer ??

Gagan 🔥🔥
Now you're thinking like a problem solver — not just a coder.

Your reasoning is **partially correct**, but there's a subtle issue.

Let's break this carefully.

🧠 Your Idea

You're saying:

Instead of:

```python
return [hashmap[required], i]
```

We do:

```python
result.append([hashmap[required], i])
```

And continue the loop.

At the end:

```python
return result
```

❗ The Problem

This **will NOT correctly count distinct pairs**.

Why?

Because of **duplicate counting**.

🚨 Example Where It Breaks

```python
nums = [1, 5, 1, 5]
target = 6
```

Let's dry run your logic.

Iteration 1

num = 1

store {1:0}

Iteration 2

num = 5
required = 1
found → append [0,1]

store {1:0, 5:1}

Iteration 3

num = 1
required = 5
found → append [1,2]

store {1:2, 5:1}

Iteration 4

num = 5
required = 1
found → append [2,3]

❌ Result:

You get:

Python Run

```
[[0,1], [1,2], [2,3]]
```

But distinct value pairs are only:

```
(1,5)
```

So answer should be:

```
1 distinct pair
```

Your method overcounts.

🔥 Important Concept: What Does "Distinct" Mean?

There are two interpretations:

1️⃣ Distinct by indices

(Count every valid combination)

2️⃣ Distinct by values

(Count unique value pairs)

Interviewers usually mean **distinct value pairs**.

✅ Correct Approach for Distinct Value Pairs

We need:

- A set to track seen numbers
- A set to track used pairs (to avoid duplicates)

🚀 Correct Implementation

Python

Run

```python
def countDistinctPairs(nums, target):
    seen = set()
    pairs = set()

    for num in nums:
        required = target - num

        if required in seen:
            pair = tuple(sorted((num, required)))
            pairs.add(pair)

        seen.add(num)

    return len(pairs)
```

🔍 Why This Works

sorted((num, required))

Ensures:

```
(1,5) and (5,1)
```

are treated the same.

🔥 Dry Run Example

Python

```python
nums = [1, 5, 1, 5]
target = 6
```

Step 1:
seen = {1}

Step 2:
5 → required=1 → pair=(1,5) → add

Step 3:
1 → required=5 → pair=(1,5) → already exists

Step 4:
5 → required=1 → pair=(1,5) → already exists

Final:

```
pairs = {(1,5)}
```

Return 1 ✅

🧠 Complexity

Time: O(n)
Space: O(n)

🔥 Now Let's Go Deeper

There are actually **3 different versions** of this problem:

1. Return one pair
2. Return all index pairs
3. Return number of distinct value pairs
4. Return number of distinct index pairs

Each requires slightly different logic.

🎯 Interview-Level Thinking

Your approach is good for:

✔ Return all index combinations

But not correct for:

✔ Distinct value pairs

🚀 Real Question For You

What if array is:

```python
nums = [3,3,3,3]
target = 6
```

How many distinct value pairs exist?

How many distinct index pairs exist?

Think carefully.

This is where mastery begins 🔥

distint value pairs that exists: 1 -> (3,3) but distinct index pairs are 2 -> (0,1) and (2,3) ?? correct ??

Gagan 🔥
This is EXACTLY where most people lose clarity.

Your **first answer is correct**.
Your **second answer is incorrect**.

Let's break it properly.

Given

Python Run

```python
nums = [3,3,3,3]
target = 6
```

✅ Distinct VALUE Pairs

We only care about unique values.

Possible value pair:

```
(3,3)
```

Even though 3 appears 4 times, the value pair is still:

```
(3,3)
```

So:

✅ Distinct value pairs = **1**

You got this right.

❌ Distinct INDEX Pairs

Now this is pure combinatorics.

We need to count all index combinations (i < j) such that:

```
nums[i] + nums[j] = 6
```

Since every element is 3:

Every pair of indices works.

So how many ways can we choose 2 indices from 4?

This is:

```
nC2 = 4C2
```

Formula:

```
nC2 = n(n-1)/2
```

So:

```
4 × 3 / 2 = 6
```

🔥 All Valid Index Pairs

Indices:

```
(0,1)
(0,2)
(0,3)
(1,2)
(1,3)
(2,3)
```

Total:

✅ Distinct index pairs = **6**

❗ Why Your Answer (2) Is Incorrect

You grouped them as:

```
(0,1) and (2,3)
```

But nothing in the problem says we cannot reuse indices across combinations.

Each pair is counted independently.

🧠 Important Concept

There are **three interpretations** of Two Sum counting problems:

Problem Type	Answer Here
Distinct value pairs	1
Distinct index pairs	6
Non-overlapping pairs only	2

Your answer (2) corresponds to:

> Maximum number of disjoint (non-overlapping) pairs

That is a different problem entirely.

🔥 Interview Insight

If interviewer says:

> "Count the number of pairs"

ALWAYS clarify:

- Do you mean distinct values?
- Or distinct index combinations?
- Or non-overlapping pairs?

This clarification shows senior-level thinking.

🚀 Now Let's Test Your Brain

If:

Python Run

```
nums = [1,1,1,2,2]
target = 3
```

Tell me:

1. Distinct value pairs?
2. Distinct index pairs?
3. Maximum non-overlapping pairs?

Take your time. This is how mastery is built.

Perfect ..... lets start with : First unique character in a string

Excellent Gagan 🔥
Now we move into **Strings + HashMap mastery**.

This is a very common interview question.

🟥 Problem: First Unique Character in a String